

Every diagram renders. Does it make sense?

The round-table promise

1 THE PROBLEM

PlantUML is being used in the organisation for creating different types of diagrams; the syntactic correctness of these diagrams can be validated using IDE plugins; however, **maintaining semantic correctness is important but challenging, and very much uncharted terrain.**

2 THE CONTRIBUTION

This roundtable invites engineers to discuss the value of introducing **linting rules for PlantUML diagrams**, helping ensure diagrams are not only syntactically correct, but also adhere to a defined semantic quality.

3 THE VALUE

Together, we will explore which conventions and quality checks deliver the most value, how diagram standards can be **integrated into engineering workflows**, and how well-structured diagrams can benefit both developers and AI-powered tools.

YOUR VIEW

Join the discussion to **share your experiences, identify common pain points**, and help define practical guidelines that improve collaboration, documentation quality, and automation across the software delivery lifecycle.

Is semantic linting of diagrams worth a PoC?

Three questions for this round table

AGREE

UNDERSTAND

DECIDE

1 The problem

Do we recognise it?

semantic correctness

common pain points

analysts & process engineers

developers & AI tools

→ a shared problem



2 The contribution

What does a semantic linter add?

linting rules

conventions

quality checks

engineering workflows

→ a shared understanding



3 The value

Is it worth trying?

documentation quality

collaboration

automation

delivery lifecycle

→ a go / no-go for a PoC



① The problem • beyond "does it render?"

What we mean by semantic correctness

Logical correctness

20%

Is it logically sound?

A block that never closes; a state machine with no starting point.

Completeness • *gaps*

30%

Is everything that should be there actually there?

A process with no end; an error path nobody drew.

Ambiguity • *ambiguities*

25%

Could two readers take it two different ways?

Arrows with no label; conditions like "sometimes"; "TBD".

Consistency

15%

One convention, here and across diagrams?

One thing with two names, kinds or stereotypes.

Traceability

5%

Can you find it, cite it, and tell who owns it?

No title, no owner, no link to the requirement.

Readability

5%

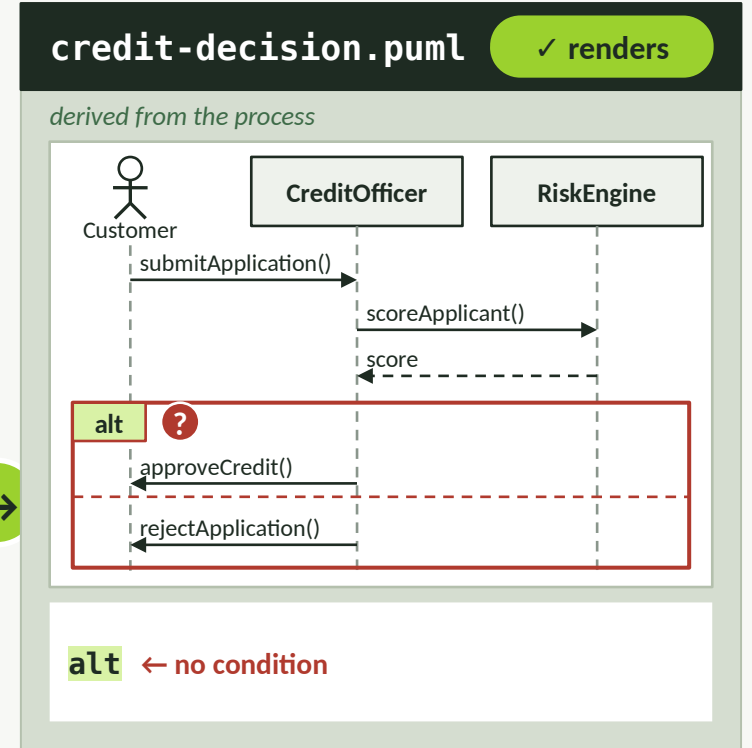
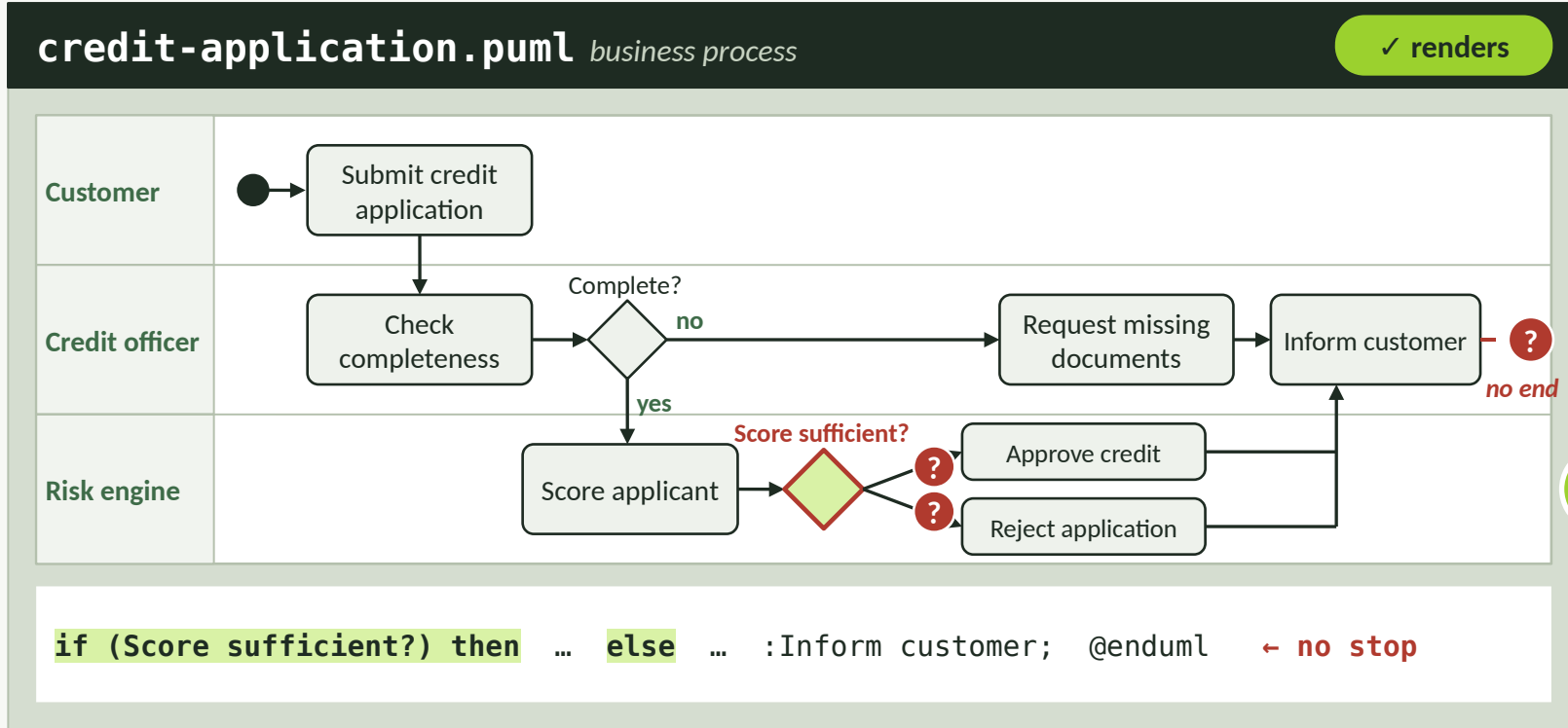
Is it small enough to take in?

Too many lifelines or messages on one canvas.

Underneath: syntax Does it render? Solved: PlantUML and its IDE plugins already check this.

- ① The problem • a credit flow: one gap, passed on to the next diagram

Both render. Which branch approves?



pumllint, linting both files

ACT003 • minor
credit-application: the decision's branches are not labelled

ACT002 • major
credit-application: the process never ends; the build fails

SEQ007 • minor
credit-decision: alt without a condition; the guess, carried down

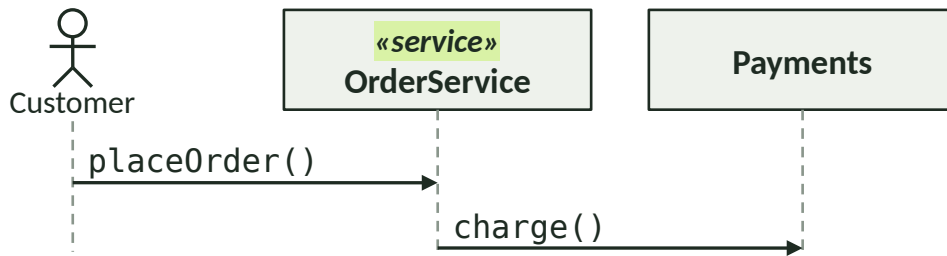
- ① The problem • each file is valid, together they contradict

Both render. Which one is right?

checkout.puml

✓ renders

```
@startuml checkout
actor Customer
participant OrderService <<service>>
participant Payments
Customer -> OrderService : placeOrder()
OrderService -> Payments : charge()
@enduml
```

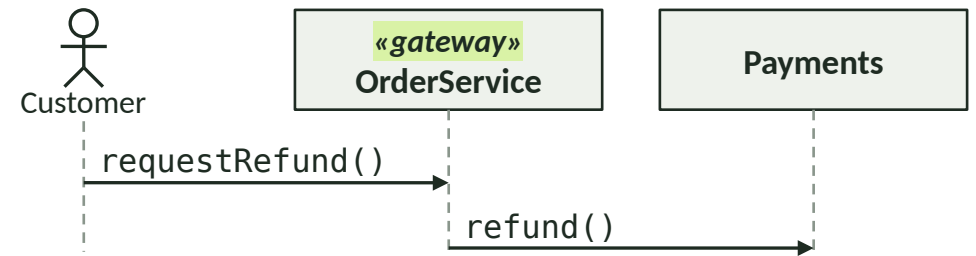


≠

refund.puml

✓ renders

```
@startuml refund
actor Customer
participant OrderService <<gateway>>
participant Payments
Customer -> OrderService : requestRefund()
OrderService -> Payments : refund()
@enduml
```

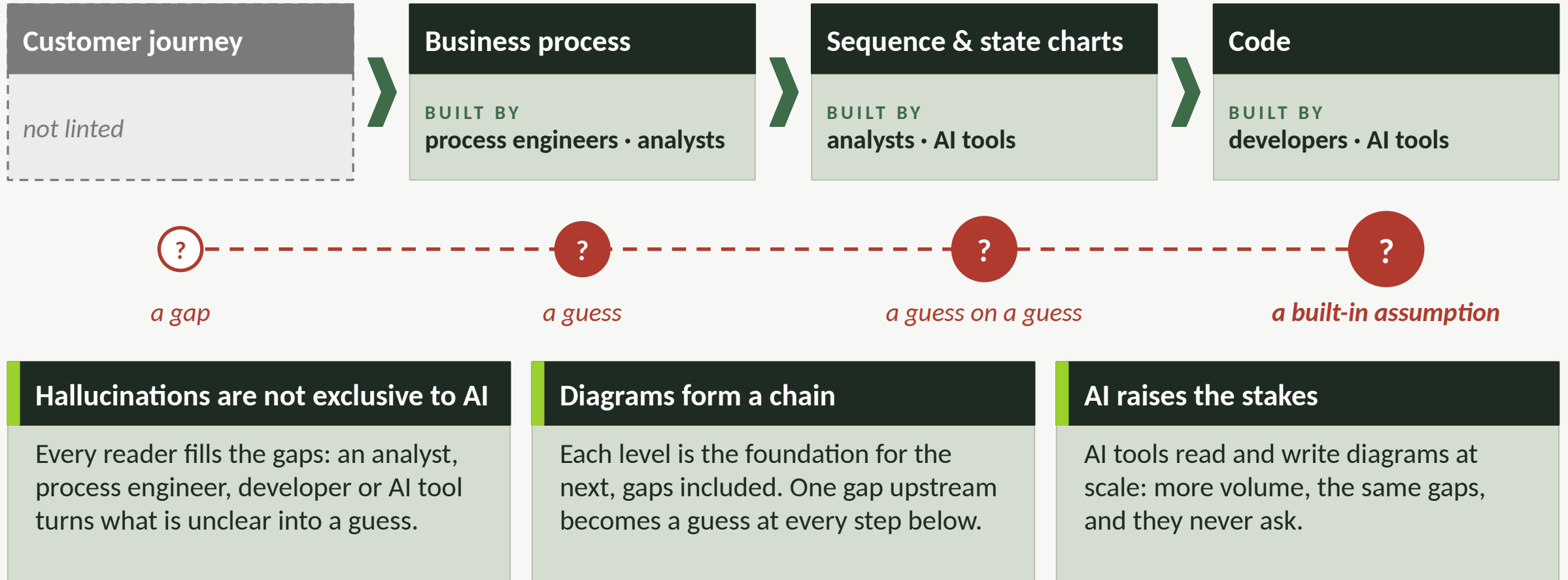


pumlLint, linting both files together

XD002 • consistency • OrderService is «service» in checkout.puml and «gateway» in refund.puml: one entity, one stereotype

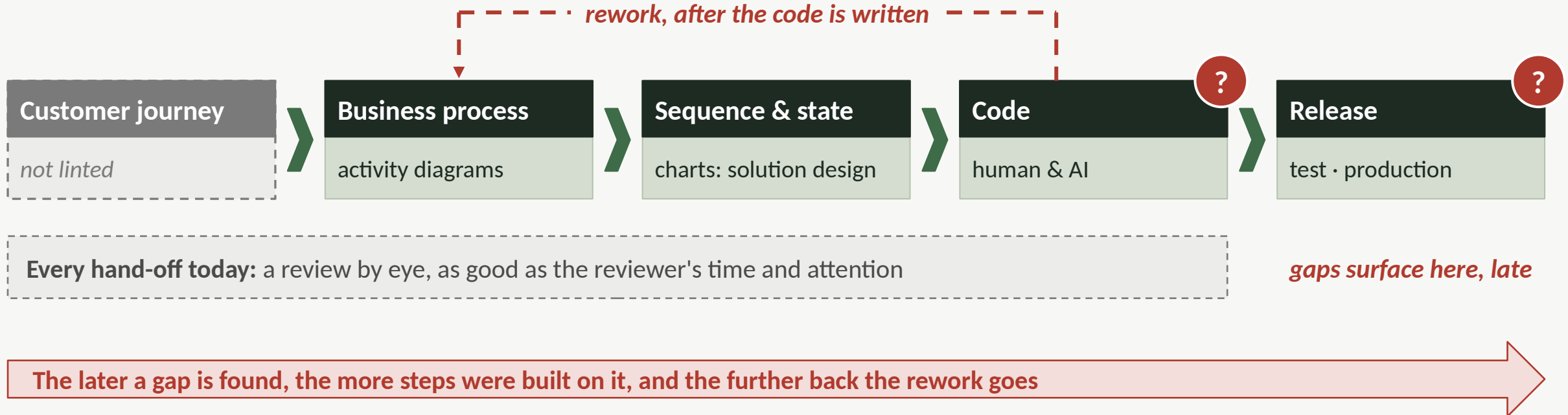
① The problem • humans and AI tools build on what the diagram says, gaps and ambiguities included

Semantic quality: why it matters now



- ① The problem • model quality is a subjective review concern

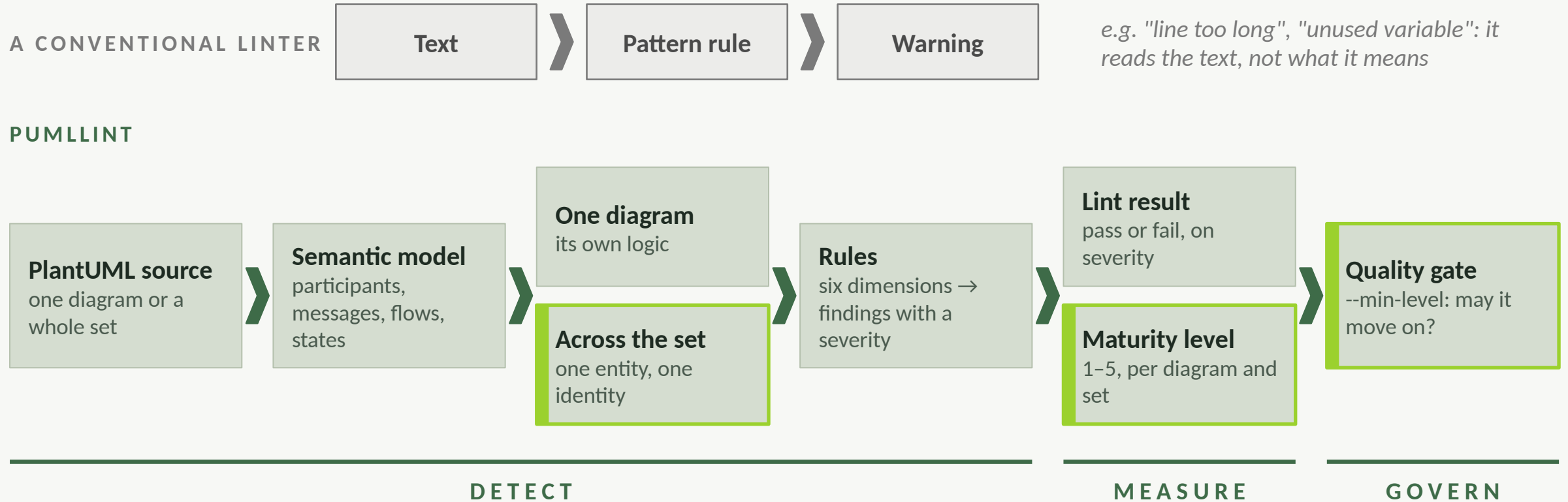
Today: reviewed by eye, found late



Question: How do you find a diagram's gaps today, and at which step?

② The contribution • a semantic static analyser for PlantUML models

From linter to static model analysis



What a conventional linter does not do. The rules stay the source of truth; the level adds them up, the gate applies your policy.

- ② The contribution • not always an error, always a reason to guess

Code smells, model smells

Code smell • SonarQube

Something in the code that makes it harder to understand, change or trust; not always a bug.

e.g. too complex • unreachable branch • unclosed resource



Model smell • pumlLint

Something in the diagram that makes it harder to interpret, implement or trust: **a reason for the next reader to guess.**

Dimension	Diagram	Model smell	Rule	Severity
Logical correctness	sequence	A participant sends messages but was never declared	SEQ001	critical
Completeness	business process	The flow never reaches an end	ACT002	major
Ambiguity	state chart	A transition with no label: what triggers it?	STA003	minor
Consistency	across the set	One participant is a service here, a database there	XD001	major
Traceability	any	No link to the requirement or decision it implements *	GEN007	minor
Readability	any	Too many elements on one canvas (over 60)	GEN009	minor

* checked once your requirement-tag format is configured

② The contribution • every finding has a severity; every diagram gets a level

Detect, measure, govern

1 DETECT findings	Severity	Meaning	Effect
	Blocker	Unusable as a specification	Fails the build • max Level 2
	Critical	Broken structure	Fails the build • blocks Level 5
	Major	Breaks a mandatory standard	Fails the build • blocks Level 5
	Minor	Breaks a recommended convention	Reported • lowers the score
	Info	Advisory	Reported • lowers the score slightly

2 MEASURE maturity	Level	Name	Read it as
	5	Method-complete	Every dimension strong, no majors: preconditions for AI code generation met (codegen profile only)
	4	Precise	Complete and unambiguous: ready for hand-off
	3	Disciplined	No blockers: ready for review, not yet for hand-off
	2	Structured	Coherent enough to work on, although it may still contain blockers
	1	Sketchy	Don't build on it

3 GOVERN	The team sets the bar, e.g. Level 2 everywhere, Level 4 before hand-off: a diagram below it does not move on. Business processes: gate on findings, not on the level; no ambiguity rule covers activity diagrams yet.
-------------	--

- ② The contribution • the level predicts what happens downstream

Does the level matter? Measured: yes

Sequence diagrams at Level 1

~1 in 3

intended behaviours failed when the generated code ran

vs

Sequence diagrams at Level 2 and up

~1 in 10

intended behaviours failed when the generated code ran

Not an opinion: AI tools generated the code, and it was run against acceptance tests written beforehand.

Robust: held across three generators from two vendors and three prompt styles.

No prompt fixes it: better prompting helped untidy diagrams, never the Level 1 ones; it cannot add guards and failure paths the diagram never had.

Scope: sequence diagrams and AI-generated code only. **Expected, not measured:** less rework and a shorter lead time; a PoC is how we would find out.

Source: github.com/fdurieux/pumlLint • [EVIDENCE.md](#), "What the product may claim" • in plain English: [docs/evidence-explained.md](#), "Verdict 1"

② The contribution • pumlLint and SonarQube, compared as linters

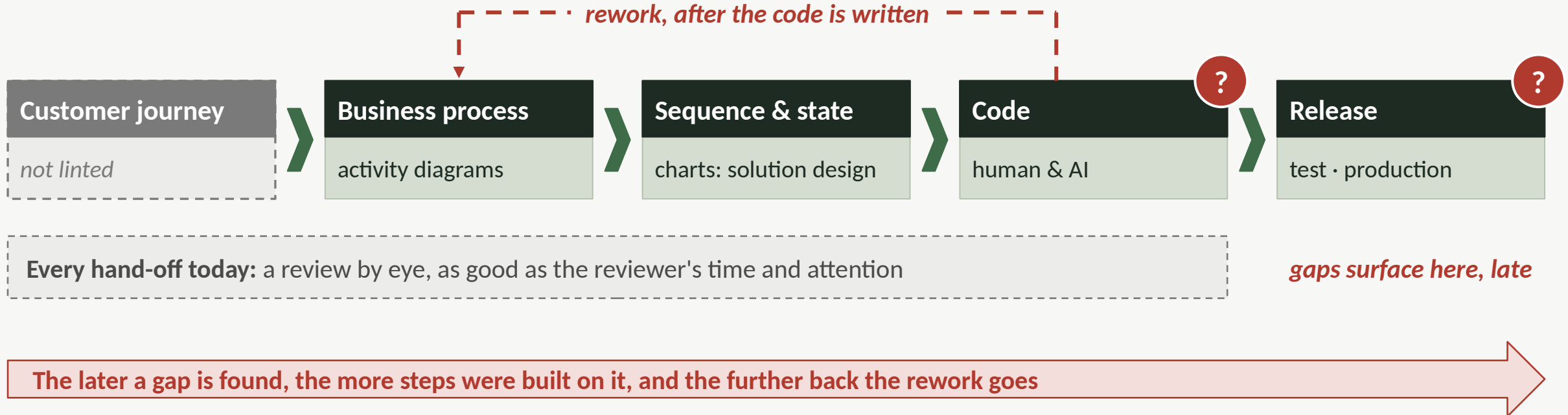
The same pattern you already trust on code

	pumlLint • on diagrams	SonarQube • on code	Where they meet
Analyses	diagram semantics, one diagram and across the set	code: control and data flow, security	pumlLint - f sonar SonarQube ✓ Findings flow in: one dashboard, PR comments ✗ SonarQube does not analyse diagrams ► The maturity gate stays in pumlLint
Detects	model smells	bugs, vulnerabilities, code smells	
Rates	maturity level 1-5	ratings A-E	
Gates	--min-level	quality gate on new code	
Existing debt	ratchet: no diagram may get worse	Clean as You Code	
Runs in	editor (LSP) • command line • CI	IDE • CI scanner • server	

Question: Which of these would you trust a tool to decide, and which stays a human review?

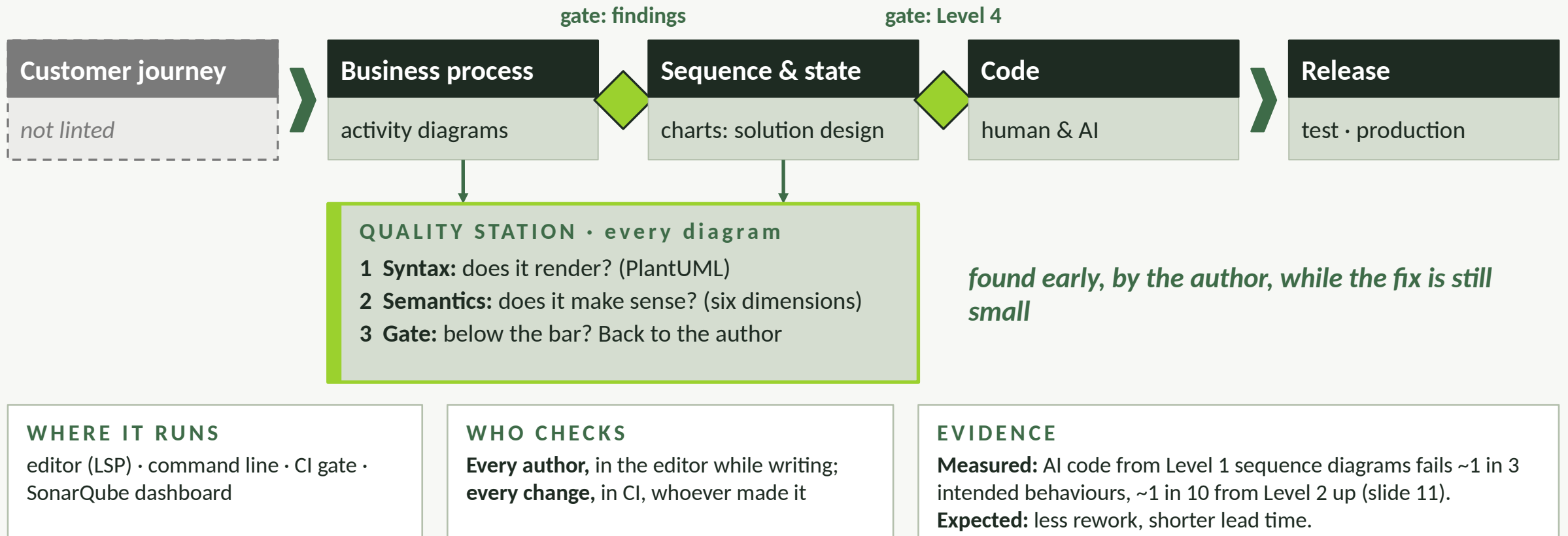
③ The value • where we started

Today: reviewed by eye, found late



③ The value • check each diagram before the next step builds on it

Shift left: checked before it is built on



③ The value • from advice to gate, step by step, without a big-bang cleanup

Diagram quality, built into the workflow

On your own machine (with or without Git)

When	How
While writing	In any editor that supports LSP: one language server, no editor-specific plugins
On demand	From the command line: check a whole folder at once, with each diagram's findings and maturity level, plus an HTML maturity report

In the delivery pipeline (diagrams stored in Git)

When	How
On every change	A CI gate checks every changed diagram, whoever authored it; the severity of its findings decides whether the build passes
From existing tools	Business processes modelled in ARIS are converted to PlantUML and pass the same gate
Over time	Dashboards (SonarQube), a maturity report, and a badge per repository showing its weakest diagram's level

Step 1 • ADVISE
report only; nothing fails

Step 2 • RATCHET
record today's levels; fail only when a diagram gets worse

Step 3 • FLOOR
require a minimum level, e.g. Level 2 everywhere, Level 4 before hand-off

Questions

- 1. What would a PoC need to show to convince you?
- 2. Where should the gate sit, and which level for which diagrams?

A PoC on semantic linting: go or no-go?

WHAT WE DISCUSSED

1 The problem	Diagrams render, but not all of them make sense; every reader downstream fills the gaps.
2 The contribution	A semantic linter finds model smells, measures maturity and gates the hand-off.
3 The value	Checked in the editor and in CI; start with advice, no big-bang cleanup.

THE VOTE



GO
run the PoC



NO-GO
not now

No-go? Ask what would change your mind, and note it.

IF GO, WE AGREE BEFORE WE LEAVE

Scope	Success criteria	Owner & timebox	Next check-in
<i>which team, which diagrams</i> ...	<i>what must the PoC show?</i> ...	<i>who leads, how many weeks</i> ...	<i>when we look at the results</i> ...

A PoC on semantic linting: go or no-go?

WHY, IN THREE LINES

1	The problem	Diagrams render, but not all make sense; every reader downstream, human or AI, fills the gaps.
2	The contribution	A semantic linter finds model smells, gives each diagram a maturity level and gates the hand-off, in the editor and in CI.
3	Measured	AI code from Level 1 sequence diagrams fails ~1 in 3 intended behaviours; from Level 2 up, ~1 in 10.

YOUR TEAM

Team / squad	...
Team contact	<i>name, role</i>
Raised by	<i>round-table attendee</i>
Date	...

Your answer	☐ Go	☐ No-go	☐ Not yet	What would change our mind?	...
-------------	-----------------------------	--------------------------------	----------------------------------	-----------------------------	----------------------------------

IF GO, WE AGREE ON

Scope	Success criteria	Owner & timebox	Next check-in
<i>which team, which diagrams</i>	<i>what must the PoC show?</i>	<i>who leads, how many weeks</i>	<i>when you look at the results</i>

Get started in ten minutes

SIX COMMANDS

```
$ pip install pumllint           # or pipx / uv tool install
$ pumllint diagrams/            # findings, with severity
$ pumllint score diagrams/      # maturity level per diagram
$ pumllint score diagrams/ -f html -o report.html
$ pumllint --list-rules         # everything it checks
$ pumllint lsp                  # findings in your editor
```

WHAT YOU GET BACK

Findings

rule, severity and line, e.g. ACT002 • major

A maturity level

per diagram and per set, with what it takes to reach the next one

An HTML report

one self-contained page, to share or attach to a review

Exit codes for CI

0 clean • 1 findings at or above --fail-on (default major) • 2 usage error

Python 3.11 or later • no runtime dependencies (PyYAML only for a YAML config). Read more in the repository: [README](#) • [docs/findings-and-scores.md](#) • [docs/rule-map.md](#) • [docs/business-processes.md](#)

Fit it to your team

Configure

pumllint.yaml (or .toml / .json) is picked up automatically:

```
rules:
  unnamed-diagram: false
  participant-naming:
    severity: major
  requirement-link:
    pattern: "REQ-\d+"
```

One exception in one diagram: ' pumllint: disable=SEQ006

Automate

GitHub Actions, pinned to a release:

```
- uses: fdurieux/pumllint@v0.33.0
  with:
    command: score
    paths: docs/diagrams
    baseline: maturity.json
    min-level: "2"
```

Or per commit: the pre-commit hooks pumllint and pumllint-score.

Connect

SonarQube

-f sonar writes SonarQube's import format: findings on the same dashboard and in pull requests.

ARIS

the separate aris2pumll tool converts processes to PlantUML; they pass the same gate.

Editors

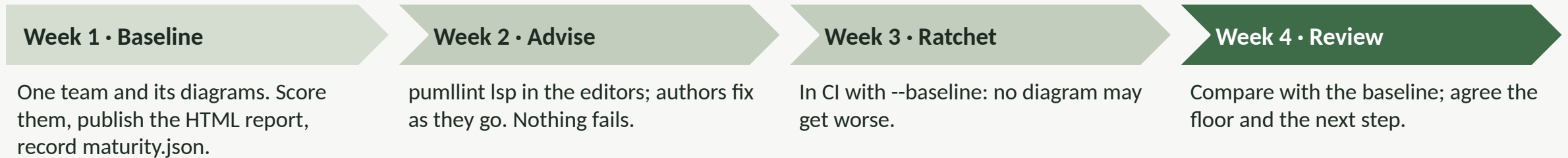
pumllint lsp in any editor that supports LSP; no editor-specific plugins.

Step 1 • ADVISE report only, no gate

Step 2 • RATCHET --baseline maturity.json

Step 3 • FLOOR --min-level 2

A four-week PoC, and what to measure



What to measure

- Findings per dimension, week 1 against week 4
- Sequence and state diagrams at Level 4; processes without a major finding
- Time to fix a finding, as the authors report it
- Noise: findings the team disagrees with, and the config changes they lead to
- Feedback from authors and reviewers
- Rework found downstream: the expected benefit, measured here for the first time

Success criteria

Agreed on the closing slide: what must the PoC show? Write them here before week 1, so week 4 compares against them.

A proposal to adapt: the length, the team and the measures are the PoC owner's call.